

Preprocessing Strategy

Amazon ML Challenge 2026 — Business Entity Resolution

Companion to EDA.pdf. It describes each preprocessing method, why it helps on this data, and how it behaves on words that appear in the test set but not in training. All figures were measured on the provided data.

Summary

- **Parse, don't just clean.** Turn each record into structured fields (core name, legal form, house number, unit, street, city, state) plus a few normalised strings. The hard negatives differ from true matches only in details that plain cleaning deletes, mainly the house number and the legal suffix.
- **For words never seen before, rules and character-level methods beat word lists.** 39% of French name tokens and 23% of French address tokens in the test set never occur in training. A method that works only through a list of known words will fail on them. Rule-based transliteration, character n-grams and phonetic keys still work.
- **Indic scripts use a closed vocabulary.** Each script has only about 165–176 distinct words, and every Indic word in the test set also appears in training. A word dictionary learned from the training pairs therefore covers Indic text completely. Rule-based transliteration with phonetic folding is the safety net: on its own it takes the median character similarity of Indic-script names to their English S1 name from 0 to 88.5, slightly above Latin-script India pairs (86.7), with no word list.
- **France is where unseen words matter.** There are no French training records, and the legal forms (SARL, SAS, EURL ...), street types (rue / r / r., av, bd, allée) and region / department names must be handled by rules, character-level similarity and statistics fitted on the test files.
- **Use dictionaries learned from training as the first layer** (Indic words, legal forms, city and state aliases), and always fall back to the rule-based path for anything not in the dictionary.
- **Measure every rule against the ground truth.** A useful rule raises field agreement on true pairs without raising it on hard negatives. Use held-out words or a held-out country to check that it generalises.

1. How different is the test vocabulary?

Tokens from the S2+S3 test files were compared with the S2+S3 training vocabulary, weighted by how often they occur.

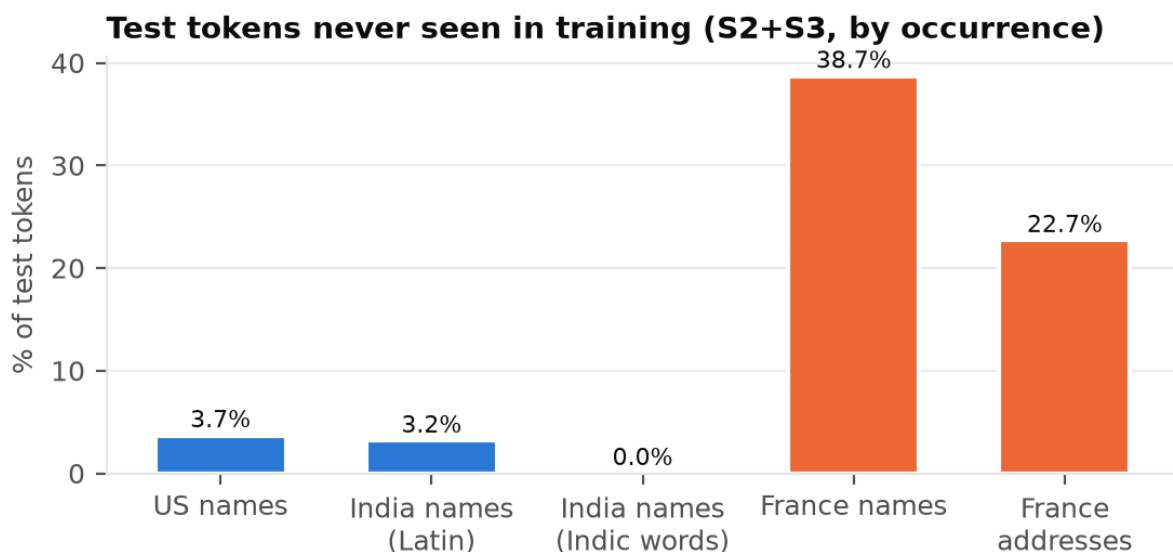


Figure 1. US and India Latin-script names are mostly known words (the unseen share is largely injected typos and random brand names). France is the real gap: about two in five name tokens are new words (école, amicale, sàrl,

établissements, Tourcoing ...).

Script (India names, word tokens)	Distinct words in train	Test occurrences	Unseen in train
Devanagari	172	1,893,431	0.00%
Telugu	169	277,176	0.00%
Kannada	167	269,869	0.00%
Tamil	166	237,809	0.00%
Bengali	164	217,429	0.00%
Gujarati	171	215,471	0.00%
Malayalam	168	135,243	0.00%
Odia	165	53,273	0.00%
Gurmukhi	176	48,620	0.00%

Of India test names that contain Indic script, 0.0% contain at least one Indic word never seen in training. Names are 93% fully in Indic script and 7% code-mixed (for example an English word followed by an Indic one).

What this means. Non-English text is two different problems. **Indic text is closed:** the data generator uses a fixed list of about 170 words per script, all of which appear in training, so a dictionary from Indic word to English word (learned by aligning true pairs) gives an exact translation for every test token. **French text is open:** about 39% of name tokens and 23% of address tokens are new, so it needs rules, character-level similarity and statistics computed on the test files. Keep the rule-based transliteration as well: it costs little and protects against any Indic word the dictionary misses.

A tokenisation pitfall found during this analysis: Python's `\w` regex does not match Indic vowel signs (matras), so `re.findall(r"\w+")` splits Indic words into syllable fragments. Tokenise Indic text on whitespace and punctuation instead.

2. Text-level preprocessing (both fields)

2.1 Safe loading and missing-value handling

What it does

Reads every file as text, so no value is silently turned into NaN. Recognises placeholder tokens inside addresses.

Why it helps

Names such as "NULL" or "NA" would otherwise become missing values. 2.5% of S2/S3 addresses contain a literal NULL, None or N/A, and 3–5% are empty. Empty addresses are 15× more common among matched records (4.4%) than among unmatched ones (0.3%), so "no address" must be modelled rather than treated as "no match".

How

- `pd.read_csv(p, sep="\t", dtype=str, keep_default_na=False, quoting=csv.QUOTE_NONE)`
- Remove NULL / None / N/A / null / nan as tokens inside addresses; add `addr_empty` and `addr_n_components` features.
- Keep the raw strings unchanged next to the parsed fields, for character-level features and error analysis.

Unseen words in test

Not word-dependent; works the same on any country.

2.2 Unicode normalisation and accent folding

What it does

Applies NFKC (unifies look-alike characters and full-width forms), then NFKD and removes combining marks. This maps é→e, à→a, ç→c.

Why it helps

The generator injects accents into English names ("Bureau of Parks", "Vidyalyaya"). French text is written both with and without accents (allée / allee, Mérignac / Merignac both appear in test). Without folding these count as different tokens.

How

- Apply to **Latin script only**. For Indic text use NFC and keep the combining marks: the vowel signs are combining characters, and stripping them destroys the word.
- Keep an accent-preserving copy for the embedding model, which can use the accents.

Unseen words in test

Fully general: works on any Latin-script word, seen or unseen. This is the main defence for French accents.

Alternatives and options

- unidecode (GPL — check the licence) or anyascii (ISC) as a broader ASCII folding, also covering ligatures such as œ→oe.

2.3 Case, punctuation and script-aware tokenisation

What it does

Lower-cases, maps &→"and" and "+"→"plus", replaces punctuation with spaces, and collapses whitespace. Tokenises Latin text with a regex and Indic text on whitespace and punctuation.

Why it helps

S2 US addresses are 93% upper-case, and names contain "and-Recreation", "CORNERSTONE-GAMING", "L.L.C.". Punctuation inside number tokens carries meaning (India "32/2/1", "B-16", "H.No"), so it must be protected.

How

- Keep "/" and "-" when they sit between digits or letter+digit; otherwise replace them with a space.
- Join dotted abbreviations before removing dots: "L.L.C." → "lIc", "S.A.R.L." → "sarI", "Pvt." → "pvt".
- Remove repeated letters only for comparison keys (see 2.6), never in the stored field.

Unseen words in test

General. The dotted-abbreviation rule covers any unseen acronym ("S.C.O.P." → "scop").

2.4 Transliteration of Indic scripts

What it does

Converts Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada and Malayalam into Latin letters, so they can be compared with the English S1 name.

Why it helps

24% of India names in S2 and 13% in S3 are in Indic script (plus native-script state names in addresses). Without transliteration their similarity to S1 is zero, and they would be missed or matched only on address.

How

- **Layer 0 (main path for this data): a learned word dictionary.** Indic text uses only about 170 distinct words per script, and every test Indic word appears in training. Align Indic and English tokens in true training pairs (same position after removing legal words, or the best character-similarity match after transliteration) and keep the most frequent English word for each Indic word (e.g. the Devanagari spellings of "private" and "limited" map back to those English words). This gives exact English words, which is better than any transliteration.
- **Layer 1 (fallback): rule-based transliteration** with indic-transliteration (MIT licence), per token, detecting the script from its Unicode block: `sanscript.transliterate(tok, SCRIPT,`

sanscript.IAST).

- **Layer 2: script-specific cleanup.** Delete the final inherent vowel ("limiteda" → "limited", common in Kannada and Telugu output), and write the nasal mark (anusvara) as "n" before a consonant ("imfotek" → "infotek").
- **Layer 3: phonetic folding**, applied to *both* sides: collapse doubled letters, w→v, sh→s, ph→f, z→j, then optionally a consonant skeleton (drop non-initial vowels: "praivet" → "prvt", "private" → "prvt").
- Apply the same method to native-script state names in addresses (1–8 forms per script, e.g. Maharashtra written in Devanagari → Maharashtra → MH).

Unseen words in test

On this data, no Indic word in the test set is unseen, so the dictionary covers everything. The rule-based layers need no vocabulary and act as insurance: if the hidden test portion contains a new word, it still gets a close Latin spelling instead of being dropped.

Alternatives and options

- Learn a character-level transliteration model (a small seq2seq or a weighted finite-state transducer) from the aligned training pairs. It captures this data's English-to-Indic spelling habits better than IAST, and still works on unseen words.
- Transliterate the *English* S1 name *into* each Indic script and compare in that script. This is useful as a second, independent view.
- Rely on a multilingual encoder (e.g. multilingual-e5-small, MIT) that already understands Indic scripts. It is good for blocking but not precise enough to be the only signal.

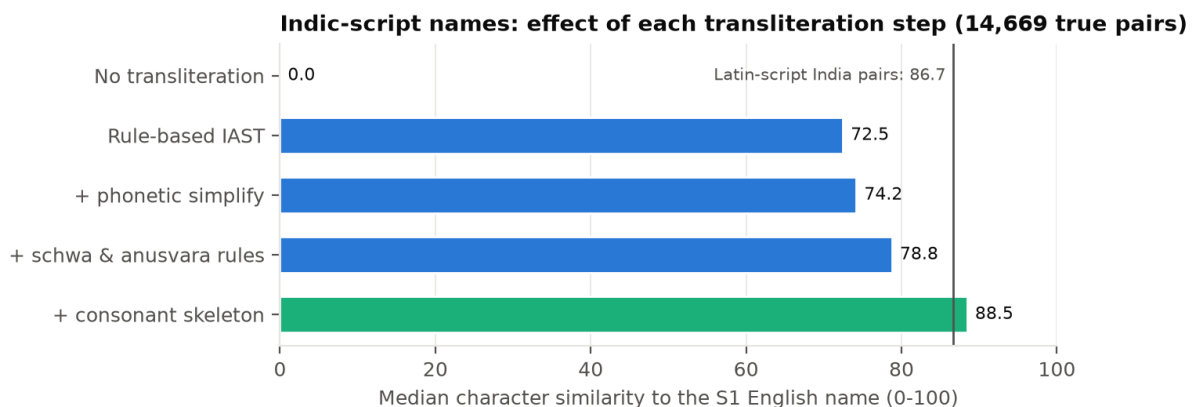


Figure 2. Median character similarity between an Indic-script S2/S3 name and its true English S1 name, after each step. Each rule adds on top of the previous one; with all of them, Indic pairs are about as easy as Latin-script India pairs (reference line).

English S1 name (folded)	Indic S2/S3 name after all layers
digital infotech private limited	dijital infotek praivet limited
gold consulting private limited	gold kansalting praivet limited
svastik energy limited	svastik enarji limited
south products private limited	saut produkts praivet limited

The remaining differences (praivet / private, enarji / energy) are systematic spelling habits. The layer-0 dictionary removes them entirely for known words; for unknown words, the consonant skeleton and character n-grams absorb them.

2.5 Typo robustness: character n-grams and phonetic keys

What it does

Represents each string by overlapping character sequences (3–4 characters) and by a phonetic code, so a few wrong letters change the representation only slightly.

Why it helps

Typos are injected on every side: "Alliatnae", "NELWKSb", "Clnton", "Mayvlile", "Pllice". 77% of distinct Latin name tokens in the test set are unseen, mostly because of these typos; by occurrence, though, they are only about 7%.

How

- TF-IDF on character 3-4-grams (analyzer="char_wb") for names and addresses. Fit IDF on train and test text together (unsupervised, using only the provided data).
- Double Metaphone for Latin-script tokens (MIT metaphone or jellyfish); the consonant skeleton for transliterated text.
- Store a name_sorted key (tokens sorted) for word-order changes ("Private Jay It Limited").

Unseen words in test

Character n-grams are the main tool for unseen words: a new word shares most of its n-grams with its misspelled or accented variants, even though the word itself was never seen.

Alternatives and options

- Typo-tolerant token matching (edit distance $\leq 1-2$ between tokens) as an alternative to n-grams for short names.
- Subword (BPE / SentencePiece) tokenisation trained on train+test text, used as input to an embedding model.

3. Business-name preprocessing

3.1 Junk prefix and suffix removal

What it does

Removes injected tokens that carry no identity: leading "--", "***", "#", "<<", "The", "Dr", "Smt"; trailing "(ID: 47290)", "| www.x.com"; and brackets around legal words ("[Limited]", "((LLC))").

Why it helps

These tokens appear throughout S2/S3 and lower both exact-match and token-overlap scores for true pairs. Together with the rest of the name normalisation (2.2, 2.3, 3.2), removing them raises the exact-match rate of names in true pairs from 3-6% (raw strings) to 43-57%.

How

- A small list of patterns, applied only at the *start* or *end* of the name, so that real words such as "Dr Reddy Labs" are not damaged in the middle.
- Learn the list from data: tokens that appear at the start or end of S2/S3 names in true pairs but not in the paired S1 name.

Unseen words in test

The patterns are symbols and position rules rather than vocabulary, so they apply to any name. For France, "Sté" and "Ets" (société, établissements) can be added as optional generic words.

3.2 Legal-suffix extraction and canonical form

What it does

Moves the legal form into its own field, legal_form, mapped to a canonical value, and removes it from name_core.

Why it helps

This is one of the two strongest precision signals: hard negatives keep the same legal form only 1.5% of the time, true pairs 65% of the time. Deleting the suffix, as generic cleaning does, throws that signal away. Keeping it out of name_core also stops "Pvt Ltd" from inflating name similarity.

How

- US: inc/incorporated, llc/l.l.c., corp/corporation, co/company, pc, pllc, lp, llp. India: pvt/private, ltd/limited, llp, "private limited" as one unit. France: sarl / s.a.r.l / sàrl, sas / s.a.s., sas, eurl, sci, sa / s.a., snc, association/asso.

- Compare as: equal / compatible (e.g. "private limited" vs "limited") / conflicting / missing on one side.
- Handle position: legal words may appear at the start ("PVT. EFS Print Ventures Ltd.", "SA PG Finance").

Unseen words in test

Legal forms are a closed set per country, so a small hand-written list per country covers the test set. For France all the variants above appear in the test data and none in training, which is why they must be added by hand.

Alternatives and options

- Group legal forms by country so that a French "SA" is not confused with an English word.
- Keep a "generic words" list (holdings, group, services, enterprises, trust) separate from legal forms, because it behaves differently.

3.3 DBA / trade-name splitting

What it does

Splits "X doing business as Y", "X dba Y", "d/b/a", "t/a", "trading as" into two names and compares each with the S1 name.

Why it helps

About 2% of S3 true pairs use this pattern (e.g. "Rizatavo Co doing business as CH Dynamic Auto Glass"). One half is usually a random brand, so comparing the whole string gives a low score for a true match.

How

- Take the maximum similarity over the parts; add a `has_dba` flag.

Unseen words in test

Works for any name. Add French equivalents (« exerçant sous le nom », « enseigne ») and Hindi ones as a small list if they appear.

3.4 Domain and handle reconstruction

What it does

Turns "firstnetworks.com", "ph0enixvidyalaya.com", "#youthunified" or "*** nathholdings.com" back into comparable text.

Why it helps

About 4% of true pairs use a domain or handle as the name. Without reconstruction they score below 50 on name similarity.

How

- Drop the TLD (.com, .in, .net, .org, .fr, .co) and the leading #, @ or ***; map look-alike digits (0→o, 1→l, 3→e).
- Compare the result with the S1 core name *with spaces removed* ("firstnetworks" vs "first networks" → "firstnetworks").
- Optionally split the joined string into words using the S1 vocabulary (word segmentation).

Unseen words in test

Comparing without spaces needs no vocabulary. Word segmentation depends on known words, so use it only as an extra feature.

3.5 Name frequency and generic-word weighting

What it does

Counts how many S1 entities share each core name and each token, and weights tokens by rarity (IDF).

Why it helps

50% of S1 entities share their normalised name with another S1 entity (for example 56 "United Engineering Private Limited"). A match on a common name or on generic words such as "services" or "enterprises" is weak evidence; a match on a rare token is strong.

How

- Fit counts on the test S1 file itself at inference time. The collisions that matter are the ones in the test set.
- Features: name_freq_s1, IDF-weighted token overlap, the rarest shared token's IDF.

Unseen words in test

IDF computed on test text gives a weight to every test word, including unseen ones. Nothing is looked up.

4. Address preprocessing

4.1 House-number and unit parsing

What it does

Extracts the street number (removing leading zeros and "#"/"##", handling "H.No", "Plot No", "S No", "Door No", "Gat No", "bis"/"ter") and, separately, the unit (Unit, Apt, Suite, Fl, Flat No, Shop No, Tower).

Why it helps

The single most important field. True pairs share the house number 87.7% of the time; hard negatives differ by exactly 1-5, 7, 9 or 11 in 62% of cases. With the naive rule (first number in the string), 11% of true pairs still show a large difference, most likely because a unit or plot number comes first. A proper parser should remove most of these.

How

- Store it as a string (e.g. "32/2/1", "b-16") and as an integer (for the difference features).
- Normalise "5 bis", "5bis", "5 b" → "5bis" (France); "011265" → "11265"; "##15140" → "15140".
- Features: equal, absolute difference, difference $\in \{1,2,3,4,5,7,9,11\}$, missing on one side, unit equal.

Unseen words in test

Numbers are language-independent, so this is the most robust signal for France. The keywords ("H.No", "Unit", "bis") come from a short list per country.

Alternatives and options

- Train a small sequence tagger (CRF or a tiny transformer) that labels address tokens as number / unit / street / city / state, using the S1 addresses (which are clean and consistently ordered) as weak labels.

4.2 Street-type abbreviation to one form

What it does

Maps street words and their abbreviations to one form per country.

Why it helps

The same street is written "Road" / "Rd" / "RD", "Place" / "Pl", "Rue" / "R" / "R.", "Avenue" / "Av" / "Av." / "Ave". In test France S2, "r" and "r." together (25% of street types) are almost as common as "rue" (37%).

How

- US: street→st, road→rd, drive→dr, avenue→ave, lane→ln, boulevard→blvd, court→ct, place→pl, circle→cir, trail→trl, highway→hwy, parkway→pkwy, terrace→ter.
- India: nagar/ngr, road/rd, opposite/opp, near/nr, colony/col, sector/sec, house number/h.no, plot/pl no.
- France: rue/r, avenue/av/ave, boulevard/bd/boul, allée/all, impasse/imp, chemin/che/ch, place/pl, route/rte, quai, cours, faubourg/fbg.

- Also: saint/st/ste, north/n, first/1st.

Unseen words in test

The street types are a closed list per country, so a hand-written table covers unseen test data. Street *names* (Maréchal Foch, Jules Verne) are open vocabulary and are handled by the character-level similarity in 2.5.

4.3 City, state and region normalisation

What it does

Maps state codes, full names and native-script names to one code, and city aliases and misspellings to one city.

Why it helps

S1 uses state codes ("MD"), S3 full names ("Maryland"), S2 native script (Maharashtra written in Devanagari). Cities appear as aliases (Poona/Pune, Calcutta/Kolkata, "City of Mayville", "Concord Township") and misspelled (Clinton, Mayvile). **In test France, S1 names the region (Hauts-de-France in 39% of S1 addresses) while S2/S3 often name the department instead (Nord, Gironde, Loire-Atlantique in about 10% each).**

How

- States: code ↔ name ↔ native script, learned from aligned training pairs (US 50 states, India 36 states and union territories).
- City aliases: count the city pairs that differ between S1 and S2/S3 in true training pairs, and keep frequent ones.
- Typo-tolerant city matching against the list of cities seen in S1 (edit distance ≤ 2 , or char n-gram similarity).
- **France department ↔ region without labels:** learn it from the test files themselves. The same city (e.g. Tourcoing) appears with "Hauts-de-France" in S1 and with "Nord" in S2/S3, so co-occurrence counts link each department to a region. This is unsupervised and uses only provided data.

Unseen words in test

The learned alias tables cover only what was seen in training. For France, use the co-occurrence method above plus typo-tolerant matching. A tiny hand-written department → region table is an alternative, but check it against the external-data rule first.

4.4 Component-order invariance

What it does

Compares addresses as sets of components rather than as ordered strings.

Why it helps

Component order is shuffled on purpose ("TX, Frisco, 15140 Elderflower Lane" vs "15140 Elderflower Lane, Frisco, TX"; "Pune, Maharashtra, ..., Sno 32/2/1"). Ordered string similarity penalises these true pairs.

How

- Split on commas, classify each component by type (number, unit, street, city, state, landmark), and compare component by component; also keep a sorted token set.

Unseen words in test

Not word-dependent.

4.5 India landmarks and partial addresses

What it does

Separates landmark phrases ("Near SBI ATM", "Opp Panjarpol", "Behind M.C Quarters", "Next to Iter College") and marks addresses that are partial.

Why it helps

7–11% of India addresses contain a landmark, and records often drop some components (the state, the locality, or everything except the city). Landmarks are useful when both sides have them, but they add noise to overall address similarity.

How

- Move landmarks into a separate field and compare them separately.
- Features: number of components on each side, share of the shorter address's tokens found in the longer one (containment rather than symmetric similarity).

Unseen words in test

The trigger words (near, opp, behind, next to, beside) are a small closed list; the landmark text itself is compared character by character.

5. Representations for blocking and matching

5.1 Blocking strings and keys

What it does

Builds, per record, the strings used by each blocking method: `name_core` (transliterated), `house_no` + `street` + `city`, and a combined "name | address" string for the embedding model; always within country.

Why it helps

Simple exact keys reach only 79.8% pair recall. Transliteration, junk removal and consistent abbreviations each raise the recall of exact and fuzzy keys, so preprocessing directly raises the recall ceiling.

How

- Exact keys: core name; house number + first street word; first name token + house number; consonant skeleton of core name.
- Fuzzy: TF-IDF character n-gram nearest neighbours; multilingual embedding nearest neighbours.

Unseen words in test

The skeleton key, character n-grams and a multilingual embedding all work on unseen words. Exact word keys do not, so never rely on them alone.

5.2 Multilingual embeddings

What it does

Encodes the combined string with a small multilingual sentence encoder (MIT/Apache, well under the 8B limit), for nearest-neighbour blocking and as a similarity feature.

Why it helps

The encoder places an Indic-script name and its English form close together and understands French, which the training data does not cover. It also catches the ~6% of true pairs whose names are unrelated, through the address.

How

- Feed it the *lightly* normalised string (keep accents and native script); the heavy folding above is for the string features.
- Optionally fine-tune it with contrastive learning on training pairs, using the hard negatives from EDA §6.

Unseen words in test

This is the strongest general method for unseen words, because the encoder was pre-trained on large amounts of multilingual text. Fine-tuning on US and India should not remove its French knowledge if learning is kept gentle.

6. Strategy for words not seen in training

Each comparison should go through a layered fallback, so a word that is missing from one layer is still handled by the next:

Layer	Handles	Works on unseen words?
1. Learned dictionaries (Indic words → English, legal forms, city / state aliases)	Known variants, with high precision; covers 100% of test Indic words	No — known words only
2. Rules (Unicode folding, transliteration, schwa / anusvara, abbreviation tables)	Systematic variation	Yes
3. Character n-grams, edit distance, phonetic codes, consonant skeleton	Typos and spelling habits	Yes
4. Statistics fitted on test text (IDF, name frequency, co-occurrence alias tables)	Word importance, France geography	Yes — computed on test
5. Multilingual embeddings	Cross-script meaning, unrelated names	Yes

How to check that it generalises, before submitting:

- **Held-out words:** remove 30% of the learned dictionary entries, rerun, and confirm that matching quality drops only slightly. That shows the rule-based layers carry the load.
- **Held-out country:** build the pipeline using US only, validate on India (and the reverse). This is the closest available test of how well it will transfer to France.
- **Parse rate on test France:** the share of France records whose house number, street, city and legal form are all filled. It should be similar to the US and India parse rates; a large gap points to a missing rule.
- **Features must not identify specific words.** Use similarity scores and flags rather than token IDs or one-hot words, and never one-hot the country, so the model transfers to unseen vocabulary.

7. Recommended order of work

#	Step	Main benefit	Check
1	Safe loading, Unicode folding, tokenisation (2.1–2.3)	Foundation	Row counts; no NaN names
2	House-number and unit parser (4.1)	Precision (hard negatives)	House number equal on >95% of US true pairs; <1% of hard negatives
3	Legal-suffix extraction, all three countries (3.2)	Precision	Equality rate: true pairs ≈65%, hard negatives ≈1.5%
4	Junk / DBA / domain handling (3.1, 3.3, 3.4)	Recall	Core name exact match on true pairs
5	Indic word dictionary + rule-based transliteration fallback (2.4, 2.5)	Recall (India)	Dictionary coverage of test Indic words (100% expected); median similarity of Indic pairs ≈ Latin pairs
6	Street / city / state normalisation, France department ↔ region (4.2–4.4)	Recall and precision	Parse rate on test France
7	Blocking strings, IDF on test, embeddings (5.1, 5.2)	Recall ceiling	Blocking recall on validation ≥97%

Implementation note: about 28M records need parsing. Run the parser once with multiprocessing over chunks (40 cores), store every derived field as parquet, and reuse it; do not parse inside the feature loop.

Appendix: measurements behind this report

- Vocabulary comparison: S2+S3 test tokens vs S2+S3 train tokens, by occurrence. Latin text tokenised with a word regex after lower-casing; Indic words split on whitespace and punctuation (India records only).

- Transliteration: all 14,669 Indic-script true pairs in the 200,000-pair EDA sample; metric = median rapidfuzz ratio between the folded S1 name and the processed S2/S3 name. Library: indic-transliteration 2.3 (MIT), IAST scheme.
- France counts: test S1/S2/S3 France records (259k / 703k / 732k).
- Hard-negative and field-agreement numbers are from EDA.pdf (code/eda.py).